The important bit here is the gemspec method, which pulls in all the dependencies and development dependencies specified in the .gemspec and adds them as though they were part of the Gemfile.

Packaging Your RubyGem

Once the gem specification is complete, you can create a packaged gem file for distribution. This is as easy as navigating to the top level of your project and typing this:

```
$ gem build aaagmnr.gemspec
  Successfully built RubyGem
  Name: aaagmnr
  Version: 0.1.0
  File: aaagmnr-0.1.0.gem
```

You'll find you now have a file called anagram-0.0.1.gem.

You can install it:

```
$ gem install aaagmnr-0.1.0.gem
Successfully installed aaagmnr-0.1.0
Parsing documentation for aaagmnr-0.1.0
Installing ri documentation for aaagmnr-0.1.0
Done installing documentation for aaagmnr after 0 seconds
1 gem installed
```

And check to see that it is there:

```
$ gem list aaagmnr -d

*** LOCAL GEMS ***

aaagmnr (0.1.0)
    Author: Noel Rappin
    Homepage: http://pragprog.com
    License: MIT
    Installed at: /Users/noel/.rbenv/versions/3.1.2/lib/ruby/gems/3.1.0

    A simple anagrams tool
```

Now you can send your gem file to friends and colleagues or share it from a server—many corporate environments use gem servers to share code. Or, you could go one better and publish it to the central RubyGems server.

Serving Public RubyGems

RubyGems.org[4] is the main repository for public Ruby libraries and projects. And, if you create a RubyGems.org account, you can push your gem file to their public servers. We're not actually going to do that here, but the command is gem push.

And, at that point, any Ruby user in the world can use it with gem install aaagmnr.

4. http://rubygems.org